

Problem 1.

Solution:

- (a) At every iteration, the algorithm selects a node v and adds it to S . Immediately afterward, Step 2(c) removes v and all nodes adjacent to v from the graph G . Therefore, no neighbor of v can ever be added to S in any later iteration, because all such neighbors have been removed before the next selection step.

Since this is true for every $v \in S$, no two nodes added to S are adjacent. Therefore we can prove S is an independent set.

- (b) We can consider the following graph: a path on 4 vertices,

$$v_1 - v_2 - v_3 - v_4.$$

All nodes have degree 1 except the two middle nodes, which have degree 2. The greedy rule chooses a node of minimum degree. So it may choose v_1 (degree 1). Which then leads to v_1 and v_2 being removed. The remaining graph contains $v_3 - v_4$, and the algorithm may choose v_3 , resulting in the independent set:

$$S = \{v_1, v_3\}.$$

This set has size 2, but the optimal independent set is

$$\{v_1, v_3\} \quad (\text{size } 2) \quad \text{or} \quad \{v_2, v_4\} \quad (\text{also size } 2).$$

To give a strictly suboptimal example, consider instead a 5-node cycle:

$$v_1 - v_2 - v_3 - v_4 - v_5 - v_1.$$

All nodes have degree 2, so the greedy algorithm may select v_1 , removing v_1 , v_2 , and v_5 . The remaining graph is just the edge $v_3 - v_4$, from which it will pick only one vertex. Therefore the greedy algorithm returns an independent set of size 2, e.g.,

$$S = \{v_1, v_3\}.$$

However, the maximum independent set has size 3, for example:

$$\{v_2, v_4, v_1\}.$$

Therefore, the greedy algorithm **does not** always return an optimal solution.

Problem 2.

Solution:

- (a) There is total budget $B = 6$ dollars and three possible investments.

For $x_i \in \{0, 1, \dots, 6\}$ we have

$$c_1(x_1) = \begin{cases} 0, & x_1 = 0, \\ 7x_1 + 2, & x_1 > 0, \end{cases}$$

$$c_2(x_2) = \begin{cases} 0, & x_2 = 0, \\ 3x_2 + 7, & x_2 > 0, \end{cases}$$

$$c_3(x_3) = \begin{cases} 0, & x_3 = 0, \\ x_3^2 - 5x_3 + 5, & x_3 > 0. \end{cases}$$

Let $f(i, b)$ represent the maximum return obtainable by investing b dollars among the first i investment opportunities.

$$f(0, b) = 0 \quad \forall b, \quad f(i, 0) = 0 \quad \forall i,$$

and for $i \geq 1$ and $b \geq 1$,

$$f(i, b) = \max_{0 \leq k \leq b} \{c_i(k) + f(i-1, b-k)\}.$$

Using the above recursion we compute $f(i, b)$ for $b = 0, \dots, 6$.

b	0	1	2	3	4	5	6
$f(1, b)$	0	9	16	23	30	37	44

b	0	1	2	3	4	5	6
$f(2, b)$	0	10	19	26	33	40	47

b	0	1	2	3	4	5	6
$f(3, b)$	0	10	19	26	33	40	47

Therefore the value of the optimal solution for budget $B = 6$ is

$$f(3, 6) = 47.$$

- (b) To recover the allocation, we trace back the choices.

From the last row, $f(3, 6)$ is attained with $x_3 = 0$, so 6 dollars remain for the first two investments.

From the second row, $f(2, 6) = 47$ is attained by investing $x_2 = 1$ dollar in Investment 2 and the remaining $6 - 1 = 5$ dollars in Investment 1.

Thus the optimal allocation is

$$x_1^* = 5, \quad x_2^* = 1, \quad x_3^* = 0,$$

with total return

$$c_1(5) + c_2(1) + c_3(0) = (7 \cdot 5 + 2) + (3 \cdot 1 + 7) + 0 = 37 + 10 = 47.$$

(c) Now we require $x_3 \leq 2$. The recursion changes only for $i = 3$:

$$f^{(\text{new})}(3, b) = \max_{0 \leq k \leq \min\{b, 2\}} \left\{ c_3(k) + f(2, b - k) \right\}.$$

Computing this for $b = 0, \dots, 6$ gives the same row as before,

$$f^{(\text{new})}(3, b) = (0, 10, 19, 26, 33, 40, 47),$$

with the maximum at $b = 6$ again obtained by choosing $x_3 = 0$. Therefore the additional constraint $x_3 \leq 2$ does not change the optimal solution:

$$x_1^* = 5, \quad x_2^* = 1, \quad x_3^* = 0, \quad \text{optimal value} = 47.$$

Problem 3.

Solution:

(a) Let the board be of size 4×7 , with the robot starting at the upper-left corner $(1, 1)$ and the destination at $(4, 7)$. The robot may only move Right (R) or Down (D).

DP function:

$$f(i, j) = \text{number of ways to reach cell } (i, j).$$

Base cases:

$$f(1, j) = 1 \quad \forall j, \quad f(i, 1) = 1 \quad \forall i,$$

because there is only one possible route if we stay on the first row or first column.

Recurrence relation:

$$f(i, j) = f(i - 1, j) + f(i, j - 1),$$

since any cell can be reached either from the left $(i, j - 1)$ or from above $(i - 1, j)$. The full DP table for a 4×7 grid is:

$i \backslash j$	1	2	3	4	5	6	7
1	1	1	1	1	1	1	1
2	1	2	3	4	5	6	7
3	1	3	6	10	15	21	28
4	1	4	10	20	35	56	84

Thus the number of possible routes from start to destination is

$$f(4, 7) = 84$$

- (b) Now assume that some cells contain coins (as shown in the problem). Let $c(i, j)$ denote the number of coins in cell (i, j) .

DP function:

$$g(i, j) = \text{maximum coins collectible on any path to cell } (i, j).$$

Base case:

$$g(1, 1) = c(1, 1).$$

First row and first column:

$$g(1, j) = g(1, j-1) + c(1, j), \quad g(i, 1) = g(i-1, 1) + c(i, 1).$$

Recurrence for all other cells:

$$g(i, j) = c(i, j) + \max(g(i-1, j), g(i, j-1)).$$

Using the coin layout in the figure, the resulting DP table is:

$i \backslash j$	1	2	3	4	5	6	7
1	0	1	1	1	2	2	2
2	0	1	1	2	3	3	3
3	0	1	1	2	3	4	4
4	0	1	2	3	4	5	5

Thus the optimal amount of coins the robot can collect is

$$g(4, 7) = 5$$

A maximizing path is:

$$R, R, R, D, R, D, R.$$

So the robot collects a total of 5 coins along an optimal route.